

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Flash Loans	Documentation quality	Medium
Timeline	2026-05-12 through 2026-05-13	Test quality	High
Language	Solidity	Total Findings	8 Fixed: 4 Acknowledged: 3 Mitigated: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0
Source Code	- flash-yields/flashyields-contracts ↗ #027f9ce ↗	Medium severity findings ⓘ	1 Mitigated: 1
Auditors	- István Böhm Auditing Engineer - Darren Jensen Auditing Engineer - Andrei Stefan Auditing Engineer	Low severity findings ⓘ	4 Fixed: 3 Acknowledged: 1
		Undetermined severity findings ⓘ	0
		Informational findings ⓘ	3 Fixed: 1 Acknowledged: 2

Summary of Findings

The FlashYields protocol implements a straightforward approval-based flash loan system with separate components for lender liquidity, vault custody, and fee verification. The core lender principal repayment invariant is generally enforced for standard ERC-20 tokens: lender balances are snapshotted before the callback and required to be at least restored afterward.

The `ApprovalFlashLoan` and `ERC20Vault` contracts are non-upgradeable. We did not find evidence that privileged roles or borrowers can drain the approved ERC20 balances of lenders through the `ApprovalFlashLoan` contract. The admin of the `ERC20Vault` contract can grant or revoke max token allowances from the vault to the `flashLoan` address configured at deployment. However, this address is immutable and cannot be changed after deployment.

The implementation uses well-established defensive patterns throughout, including snapshot-and-verify accounting, OpenZeppelin's SafeERC20 and ReentrancyGuard, role-gated administrative entry points, and Permit2 uint160 truncation guards, all of which substantially limit the attack surface in the flash loan engine itself.

The main risks identified during the audit relate to token compatibility assumptions ([ETH-1](#)), best-effort flash loan semantics ([ETH-2](#)), and operational controls ([ETH-3](#), [ETH-4](#), [ETH-5](#)). The protocol should more clearly define which token types are supported, strengthen validation and monitoring around fee/oracle configuration, and ensure that vault approval behavior matches operator expectations. We also recommend improving documentation around lender consent and vault behavior.

The security of the project would be further improved by strengthening the codebase's test suite by adding test cases to cover additional error handling and edge cases in the contracts, including testing with fee-on-transfer tokens, testing the vault as a lender, and testing additional Permit2 error-handling cases.

Fix-Review Update:

The FlashYields team has addressed the most significant issues identified during the audit. The `ERC20Vault` fee-on-transfer accounting issue ([ETH-1](#)) was fixed by crediting deposits based on the actual amount received, and the best effort lender filtering ([ETH-2](#)) was updated with the recommendations. The allowance restoration issue ([ETH-3](#)) and the lack of enforcement of the minimum deposit in the vault ([ETH-5](#)) were also resolved.

The risk of the admin keys being compromised ([ETH-4](#)) will be mitigated operationally by using a multisig. The lack of supporting fee-on-transfer token behavior natively in `ApprovalFlashLoan` ([ETH-1](#)) is treated as an acknowledged limitation, borrowers must account for transfer fees and top up deficits themselves, and this behavior should be clearly documented.

The FlashYields team also improved the contract test suite by adding test cases to cover additional error handling and edge cases in the contracts.

- Quantstamp verified that contracts were deployed correct:
- 1. FeeVerifier at 0x7F8C574610b1EA0C333E5d2681e2916403284FFf
 - 2. ApprovalFlashLoan at 0x51a6262916c1fDD5dD25E3bA11cbF76427A126A3

ID	DESCRIPTION	SEVERITY	STATUS
ETH-1	ERC20Vault Accounting and ApprovalFlashLoan Repayment Issues with Fee-on-Transfer Tokens	• Medium ⓘ	Mitigated
ETH-2	Flash Loan Best Effort Paths Can Revert on Insufficient Lender Balance	• Low ⓘ	Fixed
ETH-3	ERC20Vault.deposit Auto Grants Flash Loan Allowance	• Low ⓘ	Fixed
ETH-4	Lack of Timelock Protection for Admin Functions	• Low ⓘ	Acknowledged
ETH-5	Lack of ERC20Vault Deposit Validation	• Low ⓘ	Fixed
ETH-6	Important Functions Cannot Be Used While the Contracts Are Paused	• Informational ⓘ	Fixed
ETH-7	The verifyFees() Function Can Count Unrelated Inflows as Flash Loan Fee Payment	• Informational ⓘ	Acknowledged
ETH-8	Lack of ERC20Vault Time-Weighted Reward Balance Accounting	• Informational ⓘ	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

*i***Disclaimer**

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:

1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

- **Repo:** <https://github.com/flash-yields/flashyields-contractst> (027f9ce)
- **Files:**
 - `contracts/ApprovalFlashLoan.sol`
 - `contracts/FeeVerifier.sol`
 - `contracts/ERC20Vault.sol`

Repo: `https://github.com/flash-yields/flashyields-contracts`

Included Paths: `contracts`

Files Excluded

- All other components, such as dependencies, configuration files, deployment scripts, tests, examples, off-chain components, and contracts that are outside of the repository or were added later, were excluded from the audit.

Operational Considerations

- The contracts are intended to be deployed on the Ethereum Mainnet. On L2 networks, the FeeVerifier should also ensure that the sequencer is functioning to prevent Chainlink stale prices from appearing fresh.
- The flash loan receiver can be chosen by the adversary, as the initiator has control over the `receiverAddress` at every entry point. It is the responsibility of flash loan receivers to perform the necessary security checks inside the `executeOperation()` function, including checking the addresses of the initiator and the `ApprovalFlashLoan` contract. Any requirement that depends on the receiver being owned by the lender, pre-approved by the lender, or matched to the initiator is outside what the protocol enforces.
- Granting token transfer approvals to the `ApprovalFlashLoan` contract acts as lending consent. In addition, `ERC20Vault` deposits may make deposited assets available to the flash loan contract because the vault grants allowance to the configured `flashLoan` address.
- `Permit2 AllowanceTransfer` approvals are reusable by design. Lenders that grant long-lived Permit2 allowances to `ApprovalFlashLoan` are giving ongoing lending consent up to the approved amount and expiration, rather than signing each individual flash loan. Lenders can revoke or reduce these allowances through Permit2 if they no longer want to provide liquidity.
- The security of the protocol during runtime relies on strict governance of the privileged roles defined in the ACLManager. For instance, the `DEFAULT_ADMIN_ROLE` manages critical aspects such as fee parameters, oracle proxies, token blacklists, and the fee recipient. Similarly, the `BOOKKEEPER_ROLE` can set arbitrary USD fallback prices used for fee conversion, while the `TIMELOCK_ROLE` is responsible for updating the address of the `FeeVerifier` in `ApprovalFlashLoan`. It is essential that these roles are held by multisignature wallets, which should monitor every privileged call to ensure security.
- Off-chain operators should additionally monitor Chainlink feed freshness against the configured `maxStaleness`.
- The `flashLoan` address in `ERC20Vault` is a deployment-time commitment that cannot be migrated without redeploying the vault.
- The `ApprovalFlashLoan` contract does not support fee-on-transfer token behavior natively, the borrowers must account for transfer fees and top up deficits themselves.
- The `ERC20Vault` does not support inflationary, deflationary, or rebasing tokens because it tracks fixed per-user internal balances that do not adjust when the vault's external token balance changes.

Key Actors And Their Capabilities

ApprovalFlashLoan:

- **Admin:** Unpause the contract and update token blacklist status.
- **Pauser:** Pause the contract.
- **Timelock:** Update the address of the `feeVerifier` contract.
- **ACLManager:** Gates Admin, Pauser, and Timelock role checks.
- **Flash Loan Initiator:** Initiates a flash loan and chooses the receiver, lenders, assets, amounts, callback params, and, for Permit2 flows, permits.
- **Flash Loan Receiver:** Receives borrowed tokens and executes `executeOperation()`. Must approve principal repayment and ensure the fee recipient's balance increases by the required fee amount before verification.
- **Lender:** Supplies liquidity by granting direct ERC20 allowance to `ApprovalFlashLoan`, or by approving Permit2 and providing or maintaining sufficient Permit2 authorization.
- **FeeVerifier:** Computes fee assets and amounts, snapshots fee recipient balances, and verifies fee payment.
- **Permit2:** Used by Permit2 loan paths to set allowance from lender signatures and transfer lender funds.

ERC20Vault:

- **Admin:** Approves selected vault-held tokens to the flash loan contract with max allowance and revokes selected token approvals.
- **ACLManager:** Determines whether callers have the admin role.
- **Depositor:** Deposits ERC20 tokens into the vault and withdraws recorded balances. Deposited tokens may be lent by the vault through the configured flash loan contract if approved.
- **Flash Loan Contract:** Can pull approved vault-held tokens for flash loans and return principal to the vault.

FeeVerifier:

- **Admin:** Sets the fee recipient, default flash loan fee BPS, per-asset fee BPS, max staleness, fee token overrides, and Chainlink aggregator proxies; also clears per-asset fee BPS.
- **BookKeeper:** Sets fallback USD token prices and updates their timestamps.
- **ApprovalFlashLoan:** Calls `snapshotFees()` before the loan and `verifyFees()` after the callback.
- **Flash Loan Receiver / Borrower:** Receives fee assets, aggregated fees, fee asset count, and fee recipient through callback parameters, then arranges fee payment.
- **Fee Recipient:** Receives flash loan fees when transferred by the receiver, borrower, or another party.
- **Oracle Aggregator Proxy:** Supplies USD prices for configured tokens when an aggregator proxy is set.
- **Fee Token Holder / External Sender:** Can transfer fee tokens to `feeRecipient`; those inflows affect balance-based fee verification.

Findings

ETH-1

ERC20Vault

Accounting and

ApprovalFlashLoan

Repayment Issues with

Fee-on-Transfer Tokens

• Medium ⓘ

Mitigated

i

Update

Marked as "Fixed".

add setTokenWhitelist, use balance-delta accounting in deposit for ERC20Vault

for fee-on-transfer token, it is up to the borrower to fill the balance diff by their flash loan profit to make the tx not to revert

The balance accounting issue in the `ERC20Vault` contract was fixed. The vault only credits the amount actually received.

For the `ApprovalFlashLoan` contract, it is important to note that fee-on-transfer borrowers may receive less than the nominal amount due to fees and need to subsidize transfer taxes, including repayment-side fees, to complete the flash loan transaction. The `receiver` must account for these differences inside `executeOperation()` before returning. In particular, if the repayment transfer is taxed, the `receiver` must ensure that each lender's final balance will be restored to at least its pre-loan balance after `ApprovalFlashLoan` pulls the nominal principal amount from the `receiver` at the end of the flash loan transaction. If the `receiver` sends additional tokens directly to lenders during `executeOperation()`, it should fill only the required top-up and still retain enough balance and allowance for the contract's subsequent principal pull. Any excess sent directly to lenders is not refunded by `ApprovalFlashLoan`.

We recommend that this behavior be clearly documented, as fee-on-transfer tokens are not natively supported and successful use requires borrower-side calculation and handling.

File(s) affected: `contracts/ApprovalFlashLoan.sol`, `contracts/ERC20Vault.sol`

Description: The `ERC20Vault` contract credits users with the requested deposit amount before transferring tokens into the vault. For fee-on-transfer tokens, the actual amount received by the vault may be lower than `amounts[i]`, while `balances[tokens[i]][msg.sender]` is still increased by the full requested amount, which is then emitted in a Deposit event. This can make the vault internally insolvent and may cause later withdrawals to fail or allow early withdrawers to consume tokens that were effectively subsidized by other depositors.

A similar incompatibility exists in `ApprovalFlashLoan`. The contract transfers `amounts[i]` from lenders to the receiver, but fee-on-transfer tokens may deliver less than `amounts[i]`. During repayment, `_pullPrincipalRepayment()` attempts to pull the full `amounts[i]` back from the receiver, and `_verifyRepayment()` requires each lender's balance to be restored to its pre-loan level. For fee-on-transfer tokens, this repayment path will either revert or require the receiver to subsidize the transfer tax.

As a result, fee-on-transfer tokens cannot be reliably used for deposits or flash loans.

Recommendation: Consider using balance-delta accounting for deposits by measuring the vault's token balance before and after `safeTransferFrom()`, then credit only the amount actually received.

For flash loans, consider either explicitly rejecting fee-on-transfer tokens via a token allowlist/blacklist or redesigning the loan accounting to support balance-delta-based transfers and repayments. If fee-on-transfer tokens are not intended to be supported, document this clearly and enforce it on-chain where possible.

ETH-2

Flash Loan Best Effort Paths Can Revert on Insufficient Lender Balance

• Low ⓘ Fixed



Update

Marked as "Fixed".
The team has fixed the issue accordingly.

File(s) affected: `contracts/ApprovalFlashLoan.sol`

Description: The `flashLoanBestEffort()` and `flashLoanBestEffortMix()` flows are intended to silently skip unusable lenders and proceed with the remaining valid lenders. However, the filtering logic only checks allowance or Permit2 authorization and does not verify that the lender actually holds enough token balance.

In `_filterValidateAndSnapshotMix()`, a Permit2 lender is accepted if the lender has approved Permit2 and the Permit2 allowance can be set. The lender's token balance is snapshotted, but it is not checked against `amounts[i]`. Later, `_transferLoanedAssetsFromMemoryMix()` calls `PERMIT2.transferFrom()` outside any try/catch. If the lender's balance is below the requested amount, the Permit2 transfer reverts and aborts the entire flash loan.

The same issue exists in the direct allowance best-effort path: `_filterValidateAndSnapshot()` checks `allowance(lender, address(this)) >= amount`, but not `balanceOf(lender) >= amount`, and `_transferLoanedAssetsFromMemory()` later calls `safeTransferFrom()` without recovery logic.

A lender can therefore pass the best-effort filter through sufficient allowance or Permit2 approval, then have insufficient balance at execution time, causing the whole loan to revert. This defeats the advertised best-effort semantics and can be used to grief initiators that rely on public or third-party lender lists.

Recommendation: Consider updating the best-effort filtering to require both sufficient allowance/Permit2 authorization and sufficient lender token balance before adding a lender to the survivor arrays.

ETH-3

`ERC20Vault.deposit`

Auto Grants Flash Loan Allowance

• Low ⓘ Fixed



Update

Marked as "Fixed".
The team has fixed the issue accordingly.

File(s) affected: `contracts/ERC20Vault.sol`

Description: The `ERC20Vault` contract allows an admin to revoke a token's allowance to the flash loan contract through `revokeTokensFromFlashloan()`. However, this revocation can be immediately undone by any user depositing a nonzero amount of the same token.

In `deposit()`, if the vault's allowance to `flashLoan` is zero, the function automatically restores the allowance to `type(uint256).max`. Therefore, after an admin intentionally revokes approval for a token, any account holding even 1 wei of that token can call `deposit()` and re-enable maximum approval to the flash loan contract. This undermines the admin revocation mechanism and may expose vault-held funds to flash loan usage when the admin intended to disable it.

Additionally, this automatic allowance granting / restoration does not emit the `Approved` event, unlike `approveTokensForFlashloan()`, making it harder for off-chain monitoring systems to detect that the approval was restored.

Recommendation: Consider separating deposit logic from approval management by tracking an explicit per-token approval status controlled by admin functions. If a token has been revoked, deposits should not re-approve it unless the admin explicitly calls `approveTokensForFlashloan()`. Also emit the `Approved` event whenever approval is restored, regardless of which function performs the approval.

ETH-4 Lack of Timelock Protection for Admin Functions

• Low ⓘ Acknowledged



Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

we will use multi-sig to reduce the risk of the admin being compromised.

File(s) affected: `contracts/FeeVerifier.sol`

Description: In `FeeVerifier`, the admin can immediately update the fee recipient, default fee, asset-specific fee, maximum staleness, fee token mapping, and token oracle aggregator. For example, a compromised admin key can call `setFeeRecipient(attacker)` and redirect future flash loan fees, or call `setTokenUsdPriceAggregatorProxy()` with a malicious oracle that returns manipulated prices, causing future flash loans to pay an attacker-controlled multiple of the intended fee. This risk is especially relevant because the fee calculation depends directly on admin-configured oracle addresses and fee parameters. If a borrower or receiver contract automatically pays the fee amounts returned during the flash loan callback, malicious admin changes can result in fee overpayment, fee underpayment, denial of service, or fee redirection.

Recommendation: Consider introducing timelock protection for sensitive administrative actions, especially changes to fee recipients, fee rates, fee tokens, oracle aggregator addresses, fallback prices, and staleness parameters.

ETH-5

Lack of `ERC20Vault` Deposit Validation

• Low ⓘ

Fixed

✓ Update

Marked as "Fixed".

The team has fixed the issue by adding a minimum deposit amount for `ERC20Vault`. Deposits to the vault are not restricted by design.

File(s) affected: `contracts/ERC20Vault.sol`

Description: The `ERC20Vault` contract allows any address to deposit any nonzero token amount, including dust-sized deposits. Without proper tracking of deposits using off-chain monitoring, depositors may not receive rewards for their contributions. Moreover, an attacker could exploit the lack of validation by making numerous small deposits (e.g., as low as 1 wei), potentially overwhelming off-chain tracking systems.

Recommendation: If deposits are intended solely from specific addresses (e.g., another smart contract or a treasury), consider restricting deposit permissions to those addresses. Additionally, consider implementing a minimum deposit amount requirement and a minimum withdrawal lock duration to prevent users from generating excessive `Deposit` events.

ETH-6

Important Functions Cannot Be Used While the Contracts Are Paused

• Informational ⓘ

Fixed

✓ Update

Marked as "Fixed".

The team has fixed the issue by removing the `whenNotPaused` modifiers from the `setFeeVerifier()` and `withdraw()` functions.

File(s) affected: `contracts/ApprovalFlashLoan.sol`, `contracts/ERC20Vault.sol`

Description: The `setFeeVerifier()` function in `ApprovalFlashLoan` is protected by both `onlyTimelockRole` and `whenNotPaused`. The `whenNotPaused` modifier prevents the timelock from replacing the fee verifier while the flash loan contract is paused.

If the configured `FeeVerifier` is compromised, misconfigured, or otherwise unsafe, the appropriate response would be to pause the contract and then replace the verifier through the timelock. However, since `setFeeVerifier()` reverts while the system is paused, governance must first unpause before replacing the verifier. Otherwise, this would temporarily reopen the unsafe flash loan path before the faulty verifier has been replaced.

The `withdraw()` function in the `ERC20Vault` contract also has the `whenNotPaused` modifier. The vault's admin can prevent withdrawals by pausing the contract.

Recommendation: Consider removing the `whenNotPaused` modifier from the `setFeeVerifier()` and `withdraw()` functions.

ETH-7

The `verifyFees()` Function Can Count Unrelated Inflows as Flash Loan Fee Payment

• Informational ⓘ

Acknowledged

ⓘ Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

feeRecipient is a dedicated contract

File(s) affected: `contracts/ApprovalFlashLoan.sol`, `contracts/FeeVerifier.sol`

Description: The `verifyFees()` function verifies fee payment by checking whether the fee recipient's token balance is at least its pre-loan balance plus the expected fee amount. This check does not bind the fee payment to the flash loan receiver, borrower, or any specific transfer source.

As a result, any token inflow to `feeRecipient` between the fee snapshot and verification can satisfy the fee check. During the flash loan callback, a receiver might be able to trigger an unrelated protocol action (e.g., harvest rewards) that sends the same fee token to the fee recipient, causing `verifyFees()` to pass even if the receiver did not pay the flash loan fee from its own funds.

This is only exploitable if `feeRecipient` is a multi-purpose treasury that receives unrelated revenue, rewards, harvests, or operational transfers. In that configuration, flash loan borrowers may be able to use unrelated protocol inflows to satisfy their fee obligation, creating incorrect fee attribution.

Recommendation: Consider using a dedicated treasury for the `feeRecipient` or avoid verifying fees solely based on an unbound balance increase in a shared treasury address.

ETH-8

Lack of `ERC20Vault` Time-Weighted Reward Balance Accounting

• Informational ⓘ

Acknowledged

i

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

our reward calculation depends on the available lender balance when the flashloan happens. I think the only way the lender can game the reward is to sandwich the actual flashloan tx which rarely happen as the tx should be forwarded in the private mempool for most of the time

File(s) affected: `contracts/ERC20Vault.sol`

Description: The computation of depositor rewards may be vulnerable to sandwich attacks because `ERC20Vault` deposits and withdrawals can occur in the same or adjacent blocks. If rewards are based on a temporary balance, an attacker can deposit a large number of tokens just before the calculation and withdraw after receiving the rewards.

Please note that the reward calculation and tracking was outside the scope of the current audit.

Recommendation: Consider using time-weighted reward balance accounting with the `ERC20Vault` and enforce a minimum lock duration before `ERC20Vault` withdrawals are permitted.

Auditor Suggestions

S1 Fallback Price Cannot Be Cleared

Fixed

✓

Update
Marked as "Fixed".
The team has fixed the issue accordingly.

File(s) affected: `contracts/FeeVerifier.sol`

Description: Currently, there is no specific function to remove a stored fallback price and revert a token to the protocol's recognized missing-price state. Once a fallback price is established, it can only be overwritten or allowed to become stale.

Recommendation: Consider adding a `clearTokenUsdPrice()` function that deletes both `tokenUsdPrices` and `tokenUsdPriceUpdatedAt` values, and emits a corresponding clear event.

S2 Chainlink Oracle Integration Best Practices

Acknowledged

i

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

maxStaleness is a good enough measure

File(s) affected: `contracts/FeeVerifier.sol`

Description: `FeeVerifier` uses Chainlink price feeds to convert flash loan fees between the borrowed token and the configured fee token. While the contract checks that oracle answers are positive and not older than a global `maxStaleness`, the integration does not fully account for feed-specific Chainlink best practices and operational risks.

The contract applies one global staleness threshold to all feeds, even though Chainlink feeds have different heartbeats, deviation thresholds, market categories, and update behavior. The contract also accepts any fresh positive answer without protocol-level circuit breakers, such as acceptable price ranges, maximum deviation checks, stablecoin peg bounds, or fallback-oracle comparisons. Chainlink’s current documentation notes that `minAnswer` and `maxAnswer` are no longer used on most Data Feeds and recommends that applications implement custom circuit breakers where needed.

Recommendation: Consider configuring per-token staleness thresholds based on each feed’s heartbeat and market behavior.

Consider adding protocol-level price circuit breakers, including acceptable price ranges, deviation limits, stablecoin-specific bounds, and fallback checks where appropriate.

S3 Lack of Complete Event Coverage

Acknowledged

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

since token price will be frequently updated, skipping the event can save some gas cost

File(s) affected: `contracts/FeeVerifier.sol`

Description: The `setTokenUsdPrice()` function updates both `tokenUsdPrices` and `tokenUsdPriceUpdatedAt` but does not emit an event.

Several other privileged setters emit events that include only the new value. For example, fee recipient, oracle proxy, and fee parameter updates do not include the previous value.

While querying on-chain state is still possible, it complicates monitoring, incident response, and reconstructing historical state.

Recommendation: Consider emitting an event from `setTokenUsdPrice()` that includes the token, previous price, new price, and update timestamp.

Consider updating privileged configuration events to include both the old and new values.

S4 Missing Validation

Mitigated

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

- 1. we will double check it is a correct fee verifier contract during and after the setting
- 2. we will double check to make sure the maxStaleness is a reasonable value
- 3. we will make sure the token is correct in decimal
- 4. fixed
- 5. that is for gas saving as this will be called frequently
- 6. fixed

The team has fixed the issues `S4.4` and `S4.6` accordingly.

File(s) affected: `contracts/ApprovalFlashLoan.sol`, `contracts/FeeVerifier.sol`

Description: Several privileged configuration functions accept values that are only validated later during flash loan execution, or are not validated beyond basic, non-zero checks. This increases the risk that an admin, timelock, or bookkeeper misconfiguration causes flash loans to revert or use unintended fee-verification behavior.

- 1. **Acknowledged** In the `ApprovalFlashLoan` contract, the `setFeeVerifier()` function is missing validation that the `newFeeVerifier` is a contract.
- 2. **Acknowledged** The `FeeVerifier` contract is missing maximum validation for `maxStaleness` (e.g. `maxStaleness <= 24 hours`, see S2).

- 3. **Acknowledged** In the `FeeVerifier` contract, the `setAssetFeeToken()` function does not validate at configuration time that the selected fee token has supported decimals. Although `_tokenUnit()` later reverts if token decimals exceed `MAX_TOKEN_DECIMALS`, the misconfiguration would only be discovered during fee calculation at flash-loan time.
- 4. **Fixed** In the `FeeVerifier` contract, the `setAssetFeeToken()` function does not validate at configuration time that the required price sources are available when the fee token differs from the borrowed asset. Under the current design, the borrowed asset must have a configured aggregator proxy because `_getBorrowedTokenUsdPrice()` does not use fallback prices, while the fee token must have either a configured aggregator proxy or a nonzero fallback price. Without these checks, misconfiguration only surfaces during `snapshotFees()` at flash-loan time as `MissingTokenUsdPrice`.
- 5. **Acknowledged** In the `FeeVerifier` contract, the `setTokenUsdPrice()` function accepts `token == address(0)` and any new price without an upper bound. Consider adding an `InvalidZeroAddress` check at minimum.
- 6. **Fixed** In the `FeeVerifier` contract, the `setTokenUsdPriceAggregatorProxy()` function is missing a liveness sanity check on the aggregator. Consider verifying that the aggregator returns a valid price when setting a new price aggregator.

Recommendation: Consider applying the validations as described.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- Repo: `https://github.com/flash-yields/flashyields-contracts`
- `b3c...573 ./contracts/ApprovalFlashLoan.sol`
 - `faa...82a ./contracts/ERC20Vault.sol`
 - `071...109 ./contracts/FeeVerifier.sol`
 - `806...5dc ./contracts/LenderRegistry.sol`
 - `79a...595 ./contracts/Rebate.sol`
 - `598...036 ./contracts/libraries/InputValidator.sol`

Test Suite Results

56 tests passed, 0 test failed.

After setting up the environment variables in the `.env` file, the test suite can be run from the project directory using the following commands:

```
nvm use 24
npm install
```

```
npx hardhat test  
npx hardhat test test/e2e/Permit2Fork.test.ts --network mainnet_fork
```

Test details:

```
$ npx hardhat test
```

```
No contracts to compile
```

```
No Solidity tests to compile
```

```
Running Solidity tests
```

```
Running Mocha tests
```

```
E2E: local flash-loan flows
```

- ✓ runs **the** full direct allowance flow through FlashLoanReceiverSimple (284ms)
- ✓ runs **the** best-effort direct flow **and** skips under-approved lenders
- ✓ runs fee-**token** override flow **with** aggregator **and** fallback pricing
- ✓ uses updated fee bps **on the next direct flash loan**
- ✓ uses updated fee **token on the next direct flash loan** (56ms)
- ✓ runs Permit2 allowance flow **with** SDK-signed PermitSingle data
- ✓ runs mixed Permit2 **and** direct-allowance best-effort flow

```
- Executing deploy/001_deploy_fee_verifier.ts
```

```
- Deploying FeeVerifier with tx:
```

```
0xf29d14f9f3818effb4d2acb984bbbb6ea16f7524254325accaaeddf39548b1ca  
(type 0x2, maxFeePerGas: 4425560, maxPriorityFeePerGas: 1057579)  
=> 0x70e0ba845a1a0f2da3359c97e0285013525ffc49
```

```
txHash=0xf29d14f9f3818effb4d2acb984bbbb6ea16f7524254325accaaeddf39548b1ca  
gasUsed=1746775
```

```
- Executing deploy/002_deploy_approval_flash_loan.ts
```

```
- Deploying ApprovalFlashLoan with tx:
```

```
0xca38617f8a0cbc69a2ba27ad707f05b49e116cd678af71249f607c3f31ac4649  
(type 0x2, maxFeePerGas: 3904575, maxPriorityFeePerGas: 928821)  
=> 0x4826533b4897376654bb4d4ad88b7fafd0c98528
```

```
txHash=0xca38617f8a0cbc69a2ba27ad707f05b49e116cd678af71249f607c3f31ac4649  
gasUsed=3185755
```

```
- Executing deploy/other/deploy_flash_loan_receiver_simple.ts
```

```
- Deploying FlashLoanReceiverSimple with tx:
```

```
0xffff5fc3e324c0dfaa045ff714e253485d279fd8c1c560664a56ba0ce44f3ac1b  
(type 0x2, maxFeePerGas: 3468333, maxPriorityFeePerGas: 819478)  
=> 0x99bba657f2bbc93c02d617f8ba121cb8fc104acf
```

```
txHash=0xffff5fc3e324c0dfaa045ff714e253485d279fd8c1c560664a56ba0ce44f3ac1b  
gasUsed=925823
```

```
- Executing deploy/other/deploy_flash_loan_receiver_with_uniswap.ts
```

```
- Deploying FlashLoanReceiverWithUniswap with tx:
```

```
0xe9a2ec985c036cd1fb117979cd8f6c97676b6f66360ff4a04b65746d6d28bc43  
(type 0x2, maxFeePerGas: 3048171, maxPriorityFeePerGas: 727921)  
=> 0x0e801d84fa97b50751dbf25036d067dcf18858bf
```

```
txHash=0xe9a2ec985c036cd1fb117979cd8f6c97676b6f66360ff4a04b65746d6d28bc43  
gasUsed=1633779
```

- ✓ executes Rocketh deploy scripts locally **and** runs a deployed direct flow (149ms)

```
E2E: mainnet fork Permit2
```

- uses canonical Permit2 **for** mixed best-effort signed **and on-chain allowances**
- uses mainnet WETH **and** USDT **for** mixed best-effort signed **and on-chain allowances**

ApprovalFlashLoan

constructor **and** admin controls

- ✓ validates constructor inputs
- ✓ pauses **and** unpauses through ACL roles
- ✓ updates fee verifier through timelock role
- ✓ updates **token** blacklist through admin role

direct flash loans

- ✓ executes **a** single-lender flash loan **and** pays fees
- ✓ handles multiple lenders, assets, **and** duplicate fee assets
- ✓ reverts **on invalid inputs and insufficient allowance**
- ✓ rejects blacklisted borrowed tokens
- ✓ reverts when **the** receiver fails, underpays principal, **or** underpays fees
- ✓ blocks reentrancy **from the** receiver callback

best-effort direct flash loans

- ✓ skips lenders **with** insufficient direct allowance
- ✓ skips blacklisted tokens
- ✓ reverts when no lenders remain **after** filtering

Permit2 **and** mixed best-effort paths

- ✓ uses **an** existing Permit2 allowance **in the** all-**or**-nothing Permit2 entry point (260ms)
- ✓ sets Permit2 allowance **from** permit data when needed
- ✓ skips invalid signed rows but keeps **empty**-permit direct allowance rows **in the** mixed path
- ✓ reverts mixed path when all signed **and** direct rows are invalid
- ✓ rejects all-**or**-nothing Permit2 amounts that exceed uint160
- ✓ rejects blacklisted tokens **in the** all-**or**-nothing Permit2 path (227ms)
- ✓ skips blacklisted tokens **in the** mixed best-effort path

ERC20Vault

- ✓ validates constructor inputs
- ✓ deposits multiple tokens **and** grants flash-loan allowance
- ✓ withdraws deposited tokens **and** validates input amounts
- ✓ approves **and** revokes flash-loan allowances through admin role

FeeVerifier

constructor

- ✓ validates constructor inputs

admin setters

- ✓ updates admin-controlled fee recipient **and** default fee
- ✓ enforces configured ACL roles
- ✓ sets per-asset fee overrides **in** batch
- ✓ supports **an** explicit **0**-BPS per-asset override
- ✓ snapshots **and** verifies **zero** fees **for a 0**-BPS overridden asset
- ✓ clears per-asset fee overrides **and** restores **the** default fee
- ✓ enforces validations **and** roles **on clearAssetFlashLoanFeeBps**
- ✓ sets **max** staleness, fee-**token** overrides, fallback prices, **and** aggregators (63ms)

fee calculation **and** verification

- ✓ rounds fees up **in the** borrowed **token by** default
- ✓ converts fees **to an** override fee **token with** aggregator **and** fallback pricing
- ✓ aggregates duplicate fee assets **and** verifies balances

oracle edge cases

- ✓ requires borrowed-**token** aggregator when converting **to** another fee **token**
- ✓ rejects stale, **zero**, negative, **and** unsupported oracle prices (41ms)
- ✓ rejects stale fallback fee-**token** prices
- ✓ rejects **token** decimals above **the** supported unit conversion range

FlashLoanReceiverSimple

- ✓ stores **the** configured flash-loan contract
- ✓ rejects callback calls **from** anything other than **the** flash-loan contract
- ✓ initiates **a** direct flash loan, restores principal, **and** forwards fees

FlashLoanreceiverWithUniswap

- ✓ rejects callback calls **from** anything other than **the** flash-loan contract
- ✓ **round**-trips **two** borrowed assets through Uniswap V3 liquidity **before** repayment (43ms)

53 passing (2s)

3 pending

56 passing (56 mocha)

\$ npx hardhat test test/e2e/Permit2Fork.test.ts --network mainnet_fork

```
No contracts to compile
Running Mocha tests
```

```
E2E: mainnet fork Permit2
```

- ✓ uses canonical Permit2 **for** mixed best-effort signed **and on-chain allowances** (5521ms)
- ✓ uses mainnet WETH **and** USDT **for** mixed best-effort signed **and on-chain allowances** (4985ms)

```
3 passing (17s)
```

```
3 passing (3 mocha)
```

Changelog

- 2026-05-13 - Initial report
- 2026-05-21 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

